

# 仕様書

| 項目        | 内容                                      |
|-----------|-----------------------------------------|
| アプリケーション名 | Gido (ギド)                               |
| バージョン     | 1.0.0                                   |
| 概要        | 店舗案内（フロアガイド）およびデジタルサイネージ再生用キオスクアプリケーション |
| ターゲットOS   | Windows (x64)                           |
| 開発言語/FW   | TypeScript, React, Electron, Vite       |

## 目次

- システム構成
- 機能要件
- 画面仕様
- データ構造と設定
- 外部システム連携
- 非機能要件
- 開発・運用

## 1. システム構成

### 1.1 アーキテクチャ概要

本アプリケーションは **Electron** を採用したデスクトップアプリケーションであり、メインプロセスとレンダラープロセスの2層構造を持つ。

- Main Process (Node.js)**
  - ウィンドウ管理（キオスクモード、最前面固定）
  - ローカルファイルシステムへのアクセス（設定ファイル、ログ、キャッシュ）
  - 外部システム（ショップAPI、CMS）のポートスキャンと検出
  - 自動アップデート制御
  - IPC通信によるレンダラーとの連携
- Renderer Process (React)**
  - UI描画（フロアマップ、店舗リスト、動画プレーヤー）
  - SSE (Server-Sent Events) によるリアルタイムデータ受信
  - 設定画面の提供

### 1.2 技術スタック

- Frontend:** React 19, TypeScript, Vite, Tailwind CSS
- Backend:** Electron 39
- State Management:** React Hooks (`useState`, `useEffect`, `useContext` 等)

- **IPC:** `contextBridge` を使用したセキュアな通信
- **Log:** `electron-log`

## 2. 機能要件

### 2.1 フロアマップ表示機能

- **SVG描画:** 設定された階層（1F～4F）に対応するSVGマップを表示する。
- **現在地アイコン:** マップ上に現在位置を示すアイコン（ピンおよび吹き出し）をオーバーレイ表示する。
  - アイコンの位置、サイズ、アニメーション（フローティング等）は設定画面から変更可能。
- **画像の健全性監視:** 定期的（5分毎）に画像の読み込み状態をチェックし、破損が検知された場合は自動リロードを行う。

### 2.2 店舗リスト表示機能

- **データ取得:** 外部システム（BridgeGround）またはローカルキャッシュから店舗情報を取得する。
- **自動レイアウト:**
  - 設定された「列数」「1列あたりの行数」に基づき、店舗リストをグリッド状に配置する。
  - ジャンルごとにグルーピングし、ジャンルヘッダーを表示する。
- **ジャンル別デザイン:** ジャンルごとに定義された配色（ヘッダー文字色、背景色）を適用する。
- **情報表示:**
  - 区画番号、店舗名を表示する。
  - 店舗名が長い場合や、付加情報（`genreMemo`）がある場合、適切にフォーマットして表示する。

### 2.3 デジタルサイネージ（動画/画像再生）機能

- **CMS連携:** WonderScreen CMS (WSP) と連携し、スケジュールされたメディア（動画・画像）を再生する。
- **レイアウト:** 画面右上部の所定エリア（アスペクト比 9:16）で再生を行う。
- **フォーマット対応:** 動画（mp4, webm等）および静止画（jpg, png, svg等）に対応。

### 2.4 統合設定機能（Unified Settings）

- **アクセス方法:** 特定の操作（IPC経由など、メニューバーまたはショートカット）で呼び出し可能なモーダル設定画面。
- **プレビュー機能:** 設定変更をリアルタイムで確認できるズーム・パン可能なプレビュー画面を搭載。
- **設定項目:**
  1. **フロア設定:** 表示対象の階数選択。
  2. **レイアウト設定:** カラム数、行数、パディングの微調整。
  3. **現在地設定:** アイコンの座標、サイズ、回転、影、アニメーションの変更。
  4. **画像設定:** カスタムフロアマップ画像、その他画像の登録。
  5. **ジャンル設定:** ジャンル名（日/英）、配色のカスタマイズ。
  6. **ショップ別設定:** 特定店舗の表示カスタマイズ。

### 2.5 自動更新機能

- **GitHub Releases連携**: 新しいバージョンがリリースされた際、バックグラウンドでダウンロードし、次回起動時または手動操作時に更新を適用する。
- **パッチ画面**: 起動時に更新チェックを行い、更新がある場合は専用のパッチ画面を表示して進捗を可視化する。

### 3. 画面仕様

#### 3.1 メイン画面レイアウト (GidoApp)

画面全体を **100vh** とし、以下のように分割する。

| エリア   | 領域 (概算)               | 内容                                           |
|-------|-----------------------|----------------------------------------------|
| 上部エリア | <b>TOP_HEIGHT_VH</b>  | 左側: フロアマップ表示エリア<br>右側: サイネージ動画エリア (9:16比率固定) |
| 下部エリア | <b>LIST_HEIGHT_VH</b> | 左側: 店舗リスト表示エリア (複数カラム)<br>右側: 販促画像エリア        |

- 設定ファイル (**config/index.ts**) により比率は調整可能（デフォルトではリストエリアの高さなどで変動）。

#### 3.2 設定画面 (UnifiedSettingsScreen)

モーダルオーバーレイとして表示。

- **ヘッダー**: アプリロゴ、保存/キャンセルボタン。
- **左サイドバー**: 設定カテゴリタブ（フロア、レイアウト、現在地、画像、ジャンル、ショップ）。
- **中央エリア**: プレビュー（**react-zoom-pan-pinch** を使用した操作可能なプレビュー）。
- **右サイドバー**: 選択中のタブに応じた詳細設定フォーム。

### 4. データ構造と設定

#### 4.1 設定ファイル (**settings.json**)

アプリケーションの設定は **userData** ディレクトリ内のJSONファイルに永続化される。

```
{
  "floor": "1F", // 現在表示中のフロア
  "floorLayout": {
    "1F": { "columns": 3, "rowsPerCol": 20, "perColumnPadding": [] },
    // ...各階の設定
  },
  "locationIcons": {
    "speechBubble": { /* 位置、サイズ、アニメーション設定 */ },
    "location": { /* ピンの位置、サイズ設定 */ }
  },
  "genreMappings": {
    "ファッション": {
      "labelEn": "Fashion",
      "headerTextColor": "#00ade4",
```

```

    "rowBackgroundColor": "rgba(...)"
  }
  // ...他ジャンル
},
"imageSettings": {
  "floorMaps": { "1F": "file://..." },
  "openTimeImage": "file://..."
},
"portRanges": {
  "bridge": { "min": 8090, "max": 8099 }, // 店舗データ連携用
  "cms": { "min": 8080, "max": 8089 }    // サイネージ連携用
}
}

```

## 4.2 店舗データ構造 (Shop)

```

interface Shop {
  shopId?: string;
  name: string;      // 店舗名
  genre: string;     // ジャンル名 (グルーピングに使用)
  genreMemo: string; // 補足情報
  number: string;    // 区画番号
  floors: string[];  // 所属フロア (配列)
}

```

## 5. 外部システム連携

本アプリは、同一ネットワーク（またはローカルホスト）上の外部システムを自動検出して連携する。

### 5.1 BridgeGround (店舗管理システム)

- **プロトコル**: SSE (Server-Sent Events)
- **エンドポイント**: /api/events
- **ポート探索範囲**: 8090 - 8099 (設定可)
- **動作**: **shops** イベントを受信すると、店舗リストを更新し、ローカルキャッシュに保存する。

### 5.2 WonderScreen CMS / WSP (サイネージ配信システム)

- **プロトコル**: SSE (Server-Sent Events)
- **エンドポイント**: /api/events
- **ポート探索範囲**: 8080 - 8089 (設定可)
- **動作**:
  - **switch** イベント: 再生コンテンツの切り替え。
  - **update** イベント: タイムライン情報の更新。
  - ローカルパスが返却された場合、Electronのセキュリティ例外処理を経て **file://** プロトコルで読み込む。

## 6. 非機能要件

## 6.1 可用性・堅牢性

- **ポート自動検出:** 外部システムのポート番号が変動しても、指定範囲内であれば自動的にスキャンして接続する。
- **オフライン動作:** 起動時にAPI接続が確立できない場合、直近のキャッシュデータを使用して画面を表示する (Stale-While-Revalidate 戦略)。
- **最前面維持 (Watchdog):** 2秒ごとにウィンドウの状態を監視し、最小化されていたり背面に回っている場合は強制的に最前面・最大化に戻す (本番環境のみ)。
- **ハートビート:** 1時間ごとに稼働ログを出力する。

## 6.2 ログ管理

- **出力先:** `electron-log` を使用し、ローカルファイルに出力。
- **内容:** システム起動、ポート検出結果、APIエラー、設定変更履歴、画像の読み込みエラー等。

## 6.3 セキュリティ

- **Context Isolation:** 有効化 (`contextIsolation: true`)。
- **IPC通信:** レンダラープロセスからは、許可されたAPI (`window.electronAPI`) を通じてのみメインプロセス機能へアクセス可能。

# 7. 開発・運用

## 7.1 ビルド・デプロイ

- **ビルドツール:** `electron-builder`
- **コマンド:** `npm run electron:build`
- **インストーラー:** NSISを使用したWindows用インストーラー (.exe) を生成。

## 7.2 ディレクトリ構成 (主要なもの)

```
Gido/
├── electron/           # Main Process
│   ├── main.cjs       # エントリーポイント
│   ├── preload.cjs    # Context Bridge定義
│   └── ...
├── src/               # Renderer Process
│   ├── components/    # ShopList等
│   ├── screens/       # GidoApp, Settings
│   ├── api/           # SSEクライアント
│   └── ...
└── release/           # ビルド成果物
```

## 7.3 設定のエクスポート

`npm run export-settings` コマンドにより、現在の設定をデフォルト設定ファイルとして書き出すことが可能。